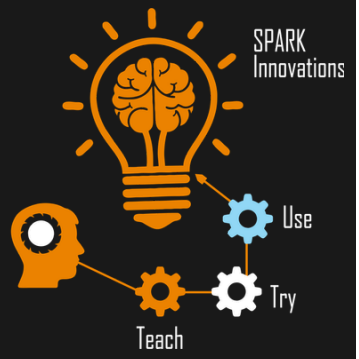
2026 SPARK ACADEMY

Mini Capstone Project Brief



Weeks 1 – 4 | Team Project | SPARK Academy 2026 | Train for Change, From Science to Practice | Due: 20th March 2026

This brief outlines **12 mini capstone projects** for your team to choose from. Each project is designed to put your Week 1–4 skills into practice: **Python fundamentals, Pandas, Matplotlib/Seaborn, OOP (Classes & Objects)**. You will work as a team, submit your code via **GitHub Classroom** using your Team GitHub account, and deliver a one-page PDF report. Code may be written as Python scripts (.py) or Jupyter Notebooks (.ipynb), either is accepted.

TEAM	PROJECT	DUE DATE	SUBMISSION
All members are required to participate.	Projects assigned, see Section 05.	20th Mar 2026 11:59 PM GMT+1	GitHub Classroom (code + PDF report).

01 | Mandatory Requirements

Every project, regardless of which one your team chooses, **must include all of the following**:

1	**CustomClasses(min.2)** Each class must have an __init__ method and at least 2 other methods. Classes must be meaningfully used — not empty shells.
2	**Standalone Functions (min. 3)** At least 3 functions defined outside of any class. These could handle data loading, cleaning, report generation, or user interaction.
3	**Pandas — Load & Clean Data** Use a CSV dataset. Load it with Pandas, handle missing values, filter or sort the data, and produce at least one summary statistic.
4	**Visualisations (min. 3)** At least 3 different chart types using Matplotlib or Seaborn (e.g. bar chart, line chart, histogram, pie chart, box plot, scatter plot). Charts must have titles and axis labels.

6

README File

A README.mdin your GitHub repo that includes: (1) what the project does, (2) step-by-step instructions on how to run it — including any libraries to install and how to execute the program, and (3) a team section listing each member's full name and the specific task they carried out (e.g. built the Patient class, handled data cleaning, created visualisations).

5

One-Page PDF Report

Submitted as part ofyour GitHub Classroom repo — push the PDF alongside your code before the deadline. See Section 03 for the required structure.

7

GitHub Repository

All codesubmitted via the GitHub Classroom assignment link that will be shared with your team. Your code can be written as Python scripts (.py) or Jupyter Notebooks (.ipynb) — either is accepted. Each team must accept the assignment using their Team GitHub account and push all final code before the deadline.

02 | The 12 Projects

Each team has been **assigned a project** — see Section 05 for your team's assignment. The project details below describe what each project involves, the classes and functions to build, the dataset provided, and the visualisations expected. Teams assigned to the same project are **strictly forbidden** from sharing code or collaborating — your implementation must be entirely your own.

#01 Medical Symptom Checker & Report Generator

Users type symptoms into the terminal. A SymptomChecker class matches them against a CSV of conditions, calculates a risk score, and ranks possible diagnoses. A ReportGenerator class prints a summary and saves a bar chart of the top diagnoses ranked by likelihood score.

Classes: SymptomChecker, ReportGenerator

Functions: load_data(), calculate_risk_score(), display_menu()

Data: symptoms_conditions.csv (provided)

Charts: Bar chart (diagnosis likelihood), pie chart (symptom categories), histogram (risk scores)

#02 Patient Records Management System

A full command-line system to add, search, filter, and analyse patient records. A Patient class stores individual records, and a RecordsManager class handles the full database loaded from CSV. Generates summary charts and lets users filter by diagnosis, age group, or gender.

Classes: Patient, RecordsManager

Functions: load_records(), search_patient(), generate_summary()

Data: patient_records.csv (provided)

Charts: Bar chart (diagnoses), histogram (age distribution), pie chart (gender split)

#03 Hospital KPI Dashboard

Load hospital operations data from a CSV. A Department class holds data for each ward, and a Hospital class aggregates the full picture. Compute KPIs: average wait time, bed occupancy rate, scan utilisation. Output a multi-panel Seaborn dashboard saved as a PNG file.

Classes: Department, Hospital

Functions: load_data(), compute_kpis(), save_dashboard()

Data: hospital_ops.csv (provided)

Charts: Bar chart (wait times), heatmap (bed occupancy), line chart (daily scan volume)

#04 Disease Outbreak Tracker

Trackdisease case counts across African countries over time. An OutbreakTracker class manages all records; a Country class stores per-country data. Load real or simulated WHO-style data, detect surges, flag countries above a threshold, and produce trend and comparison charts.

Classes: Country, OutbreakTracker

Functions: load_outbreak_data(), detect_surge(), generate_alert()

Data: outbreak_data.csv (provided)

Charts: Line chart (cases over time), bar chart (country comparison), scatter plot (cases vs population)

#05 Medical Chatbot (Command-Line)

Aterminal-based chatbot that answers basic medical questions. A KnowledgeBase class loads Q&A pairs from CSV and handles keyword matching. A Chatbot class manages the conversation loop, tracks session history, and ends each session with a bar chart of the most frequently asked topic categories.

Classes: KnowledgeBase, Chatbot

Functions: load_knowledge_base(), match_question(), save_session_log()

Data: medical_qa.csv (team-built with 40+ Q&A; pairs)

Charts: Bar chart (topic frequency), pie chart (query categories), line chart (questions per session)

#06 Drug Inventory & Expiry Manager

Apharmacy management tool using Drug and Inventory classes. Load stock data from CSV, flag expired or low-stock medications, print restocking alerts, and visualise current inventory. Practical and directly relevant to African clinic and hospital settings.

Classes: Drug, Inventory

Functions: load_stock(), check_expiry(), generate_restock_report()

Data: drug_inventory.csv (provided)

Charts: Bar chart (stock levels), pie chart (drug categories), line chart (stock changes over time)

#07 BMI & Health Risk Calculator

Batchprocess a CSV of patient records. A Patient class stores health data; a HealthAssessment class calculates BMI, blood pressure category, and overall risk level. Output individual assessments and population-level distribution charts using Seaborn.

Classes: Patient, HealthAssessment

Functions: load_patients(), calculate_bmi(), categorise_risk()

Data: patient_health.csv (provided)

Charts: Histogram (BMI distribution), bar chart (risk categories), scatter plot (BMI vs age)

#08 Blood Bank Management System

Track blood donations and patient requests using Donor and BloodBank classes. Load donation history from CSV, check supply vs demand per blood type, flag critical shortages, and generate visualisations of inventory and donor demographics.

Classes: Donor, BloodBank

Functions: load_donations(), check_supply(), flag_shortage()

Data: blood_donations.csv (provided)

Charts: Bar chart (blood type inventory), pie chart (donor demographics), line chart (donations over time)

#09 Clinical Trial Data Analyser

Loadaclinical trial CSV and split patients into treatment and control groups. A Trial class manages the full dataset; a PatientGroup class handles per-group stats. Compute means, standard deviations, and outcome comparisons. Visualise results with box plots and bar charts.

Classes: Trial, PatientGroup

Functions: load_trial_data(), compute_group_stats(), visualise_outcomes()

Data: clinical_trial.csv (provided)

Charts: Box plot (outcome distributions), bar chart (group averages), histogram (patient age spread)

#10 Medical Quiz Application

Aninteractive terminal quiz engine built around medical imaging topics covered in SPARK. A Quiz class manages rounds; a Question class holds content loaded from CSV; a ScoreTracker class records performance. End each session with a performance chart.

Classes: Quiz, Question, ScoreTracker

Functions: load_questions(), run_round(), display_results()

Data: quiz_questions.csv (provided)

Charts: Bar chart (score per category), line chart (score trend over sessions), pie chart (correct vs incorrect)

#11 Scan Scheduling & Queue Manager

Simulate a hospital scan department managing MRI, CT, and X-Ray queues. Patient and Scan classes store individual records; a Scheduler class manages the queue and prioritises by urgency level. Track wait times and visualise utilisation across a simulated working day.

Classes: Patient, Scan, Scheduler

Functions: load_patients(), prioritise_queue(), calculate_wait_time()

Data: scan_requests.csv (provided)

Charts: Bar chart (queue by scan type), line chart (wait times across day), pie chart (urgency breakdown)

#12 Maternal & Child Health Tracker

LoadaCSV of maternal health records and identify high-risk pregnancies based on clinical thresholds. A Mother class stores patient data; a HealthWorker class manages assessments and flags alerts. Generate risk distribution charts and a summary report.

Classes: Mother, HealthWorker

Functions: load_records(), assess_risk(), generate_alert_report()

Data: maternal_health.csv (provided)

Charts: Histogram (age distribution), bar chart (risk levels), scatter plot (BP vs risk score)

03 | One-Page PDF Report Structure

Every team must submit a **one-page PDF report** alongside their GitHub repo. The report must follow this structure exactly. Keep it concise — one page only.

1. **Project Title & Assigned Team**
   State the project title and your assigned team name.

2. **Project Overview**
   2–3 sentences describing what your program does, the problem it addresses, and who would use it in a real health setting.

3. **Technical Summary**
   Briefly describe the classes you built, the key functions, how you used Pandas to process the data, and what the visualisations show.

4. **Challenges & How You Solved Them**
   Describe one or two real technical challenges your team faced during the project and the steps you took to resolve them.

5. **README & Team Contributions**
   Confirm that your README is in the GitHub repo. The README must include: what the project does, step-by-step instructions on how to run it (libraries to install, how to execute), and each team member's full name with the specific task they carried out.

04 | Grading Rubric

Projects are graded out of **100 marks** across six criteria. The same rubric applies to all 12 projects so scoring is consistent across teams.

CRITERION	MKS	FULL MARKS (ALL REQUIRED)	PARTIAL CREDIT	MINIMAL / NONE
OOP & Classes	**25**	2+ classes with meaningful methods, clean __init__, and classes used throughout the program.	Only 1 class present, or methods are mostly empty shells.	No classes written, or classes exist but are never used.
Functions & Logic	**20**	3+ standalone functions outside classes, each with a clear purpose, parameters, and return values.	Only 1–2 functions, or the logic inside functions is unclear or incomplete.	No standalone functions written outside of classes.
Pandas — Data Handling	**20**	CSV loaded correctly, missing values handled, data filtered or sorted, and summary statistics computed.	CSV is loaded but little to no cleaning, filtering, or analysis is applied.	Pandas not used at all, or the dataset is never meaningfully processed.
Visualisations	**20**	3+ different chart types produced, all with titles and axis labels, charts are readable and relevant to the project.	Only 1–2 charts produced, or charts are missing titles and labels.	No charts produced, or charts are blank and empty.
README & GitHub	**10**	Clear README with project description, step-by-step run instructions, and a team section listing each member's name and their task. Code submitted via GitHub Classroom using the Team GitHub account.	README present but missing run instructions or team section, or submission is incomplete.	No README file, or assignment not submitted via GitHub Classroom.
PDF Report	**5**	Follows the 5-section structure exactly, fits on one page, clearly written, and pushed to the GitHub Classroom repo.	Missing 1–2 required sections, or the report exceeds one page.	Report not submitted or not pushed to the repo.
TOTAL				**100 marks**

A Note on Collaboration & Integrity: Working as a team on this project builds the collaborative skills that will define you as a health tech professional. It prepares you for the group projects and grand competition ahead. Write your own code — because the skill you build here is yours to keep long after the programme ends. Teams sharing the same project must not share code or implementation details. Your GitHub commit history will be reviewed, and any evidence of copying will result in a zero for all teams involved.

05 | Team Project Assignments

Projects have been **randomly assigned** to all 33 teams. **9 projects have 3 teams** and 3 projects have 2 teams (33 teams across 12 projects). Your team's assignment is final — find your team name in the assignment table below.

#	PROJECT TITLE	ASSIGNED TEAMS
01	**Medical Symptom Checker & Report Generator**	Team Egypt · Team Kenya · Team Morocco
02	**Patient Records Management System**	Team Nigeria Lagos II · Team Nigeria North West · Team Wakanda
03	**Hospital KPI Dashboard**	Team Ethiopia · Team Nigeria UNN · Team Uganda
04	**Disease Outbreak Tracker**	Team Gambia · Team Nepal · Team Senegal
05	**Medical Chatbot (Command-Line)**	Team Nigeria Minna · Team Nigeria South West
06	**Drug Inventory & Expiry Manager**	Team Ghana Tamale · Team Nigeria Abuja · Team Tanzania
07	**BMI & Health Risk Calculator**	Team Nigeria Lagos I · Team Zambia
08	**Blood Bank Management System**	Team Nigeria Kano · Team Nigeria South South · Team South Africa
09	**Clinical Trial Data Analyser**	Team Nigeria North East · Team Rwanda · Team Zimbabwe
10	**Medical Quiz Application**	Team Benin · Team Nigeria North Central · Team Nigeria Oyo
11	**Scan Scheduling & Queue Manager**	Team Ghana Accra · Team Ghana Kumasi · Team Nigeria Newgate
12	**Maternal & Child Health Tracker**	Team Botswana · Team Cameroon